VC3

驗證：怎麼知道 AI 寫的是對的

VC3 — Verification: How Do You Know It's Right?

生成從來不是瓶頸，判斷才是。
這一份是全課程的中軸——沒有這一週，前面兩週學到的只是「做得出東西」。

Vibe Coding 專班 · 第 3 週 · 混合程度班 | 謝楠楨

這一週結束時，你手上會多出三樣東西

1 一份審查檢核表

- 你會知道 AI 寫的那段程式碼在做什麼
- 八個檢查項目，逐項打勾並寫下判斷
- 不需要會寫程式，只需要會問對問題
- 幾乎一定會找到至少一個問題
- 找到的那個問題，就是你這週最大的收穫

2 一份會自己跑的測試

- 雙擊 test.html 就會執行，顯示綠色或紅色
- 不需要安裝 npm，不需要開終端機
- 把第 1 週訂的驗收條件變成可重複執行的檢查
- 以後每改一次程式，重跑一次就知道有沒有改壞
- 本週會做到 29 項全綠

3 一份驗證報告

- 測了什麼／怎麼測／抓到什麼／還有哪些不確定
- 第四段最重要，而且不能留白
- 期末專題的五項交付物之一
- 評分裡佔比最高的一項（14%）
- 說得出自己不知道什麼，才叫真的驗證過

注意這三樣東西的順序：先「讀懂」，再「測試」，最後「說出不確定」。跳過第一步直接寫測試，你會測到錯的東西還不知道——因為你根本不知道它在算什麼。

本週地圖：哪幾頁要讀、哪幾頁要動手

頁	段落	怎麼用	預估時間
4–8	PART 01　為什麼驗證是這門課的中軸	讀。建立觀念，不必動手	10 分
9–15	PART 02　讀懂 AI 產出的程式碼	讀 + 記下第 15 頁的檢核表	20 分
16–21	PART 03　什麼是測試	讀。不需要學會寫測試，只要看得懂	20 分
22–26	PART 04　你以為測完了	讀。第 23 頁的網格等一下要用	15 分
27–46	PART 05　動手做（Step 1–7）	打開你自己的專案，跟著做。這是本週的主體	90–120 分
47–48	卡住了？照症狀查	不必從頭讀，卡住時再翻	—
49	常見的四個誤解	做完之後回頭看一次	5 分
50–52	自我檢核、本週交付與回顧	交作業前一定要看	10 分

自學建議：觀念段（4–26 頁）一次讀完約 65 分鐘；動手段（27–46 頁）建議分兩次做，中間休息。全部做完約 3 小時。不要想一口氣做完——Step 1 的「讀懂」需要慢慢來。

PART 01

為什麼驗證是中軸

Why Verification Is the Core

前兩週你已經能做出東西了。

這一週要回答的是另一個問題：你怎麼知道它是對的?

先看一個真實的情況：這個試算器「跑起來沒問題」

身分別 ＼ CMS	CMS 2 級	3 級	4 級	5 級	6 級	7 級	8 級
一般戶	○	?	?	○	?	?	?
中低收	?	?	?	?	?	?	?
低收	?	?	?	?	?	?	?

○ 測過，結果正確　　× 測過，發現錯誤　　? 還沒測過

你做完 v0.2 之後的狀態

畫面出得來、按鈕會動、數字有跑出來
第 2 週的三個錯也修好了
看起來完全沒問題

但你實際只驗過兩格

自己最熟的那兩種組合
剩下 19 格是「假設它會對」
而假設不是證據

為什麼這很危險

算錯的金額不會報錯、不會變色、不會提醒你
它會用跟正確答案一模一樣的樣子顯示出來
使用者也不會發現——他們正是因為不知道才來用的

這一週要把？ 全部變成 ○ 或 ×

並且說得出「我怎麼知道它是對的」
這是本課程對「做完了」的定義

這一頁不是要嚇你。相反地： 21 格點一點只要三分鐘。真正的問題從來不是「驗證很難」，而是「沒有人告訴你要驗證什麼」——這一份講義就是在回答這件事。

你不會寫程式，但你會判斷對錯

	× 從程式碼找答案（你的弱項）	○ 從專業知識找答案（你的強項）
低收戶的自負額	看程式碼 → rate: 0.05 → 看起來很合理 → 看不出問題	查給付基準 → 低收是 0% → 對照畫面上的 1,205 → 抓到了
第 8 級的給付上限	看程式碼 → Math.min (level, 7) → 語法完全正確 → 看不出問題	查給付基準 → 第 8 級應為 36,180 → 實際顯示 32,090 → 抓到了
「僅供參考」這行字	看程式碼 → 只是一段文字 → 跟程式無關 → 不會注意到	想使用情境 → 家屬會不會當成核定金額 → 措辭要不要再明確 → 抓到了

這是這門課最重要的一句話：抓錯不需要看得懂程式碼，需要的是「知道正確答案應該是什麼」。你的長照專業知識，在這件事上比工程師強得多——他看得懂每一行語法，但他不知道低收入戶的部分負擔是 0%。

驗證的三層：由下往上，缺一不可

第三層　說出不確定

我知道自己還有哪些沒驗到，並且寫下來

- 驗證報告第四段
- 已知的限制
- 什麼情況不該用它

第二層　測試

把驗收條件變成可重複執行的檢查

- test.html
- 29 項
- 改完重跑一次

第一層　讀懂

我知道這段程式碼在做什麼、在什麼情況下會錯

- 逐行加註解
- 審查檢核表
- 問邊界問題

多數人只做到第二層

有測試、全綠、就交出去。但沒說出「哪些沒測到」——而使用者最需要知道的正是那些

只做第二層的風險

測試會給你一種「已經驗過了」的安全感。實際上它只證明了「跟你想的一樣」

跳過第一層的後果

你會照著自己誤解的邏輯寫測試。測試全綠，金額還是錯的

本週的三個交付物

審查檢核表（第一層）、test.html （第二層）、驗證報告（第三層）。一層對應一樣

為什麼順序不能反過來：測試檢查的是「跟你的期望一致」，不是「期望本身正確」。如果你沒先讀懂它在算什麼，你寫出來的測試也會照著錯誤的邏輯去驗——測試全綠，結果還是錯的。

為什麼 AI 不會主動告訴你它錯了

① 它不知道自己錯了

- 模型輸出的是「最可能的下一段文字」，不是「經過驗證的答案」
- 它沒有執行過那段程式碼，也沒有查過長照給付基準
- 它的自信程度跟正確與否沒有關聯——這是 M7 講過的「過度自信」在程式碼上的版本

② 你問「這樣對嗎」，它幾乎一定說對

- 直接問「有沒有問題」，得到的通常是「看起來沒問題」
- 要問得具體才有用：「如果 identity 是空字串會發生什麼事？」「第 8 級走到哪一行？」
- 更好的問法：「請列出這段程式碼做了哪些假設」——假設攤開來，錯誤就浮出來了

③ 「跑得起來」跟「算得對」是兩件事

- 語法錯誤會報錯，邏輯錯誤不會。而你遇到的幾乎都是後者
- 第 2 週的三個 bug 裡，只有一個會在 Console 留下痕跡
- 另外兩個安安靜靜地顯示了錯的金額——這才是常態

換個角度想： AI 就像一位打字非常快、但完全不懂長照的實習生。你不會讓這樣的人直接把試算結果發給家屬——你會先看過。這一週教的就是「怎麼看」。

PART 02

讀懂 AI 產出的程式碼

Reading Code Without Writing It

你不必學會寫程式。

但你必須看得懂它在算什麼，否則永遠只能相信它。

四個技巧，都不需要語法知識

1 要它逐行加註解

- 最有效、也最容易上手的一個
- 用中文、用你聽得懂的話解釋每一行
- 看不懂的地方直接追問「這句是什麼意思」
- 本週 Step 1 就是做這件事
- 讀完你會發現：其實沒幾行

2 要它畫出流程

- 「請把這段的執行順序用文字流程圖畫出來」
- 適合有分支、有條件判斷的程式碼
- 流程圖看得懂，就知道哪些情況會走到哪裡
- 也能看出「有沒有漏掉某種情況」
- 例：沒選身分別時走到哪？

3 問它邊界問題

- 「輸入是空值、負數、超出範圍時會怎樣？」
- 這一問常常會逼出它自己也沒想到的漏洞
- 它會老實回答「會丟出錯誤」或「會回傳 undefined」
- 而那正是你要找的東西
- 問完記得自己實際試一次驗證

共同點：這四個技巧都是「把程式碼翻譯成你的語言」，而不是「把你變成程式設計師」。你要判斷的是邏輯與領域知識，那兩件事你本來就會。

技巧 ①　要它逐行加中文註解

可以直接照抄的指令

① 先貼這一段

請把 `calculator.js` 的 `calculate` 函式逐行加上中文註解，
用完全不懂程式的人也看得懂的話解釋每一行在做什麼。
不要改動任何邏輯，只加註解。

看完它的回答之後，接著問這一段

② 追問

第 N 行「〔貼上你看不懂的那一行〕」是什麼意思？
請舉一個實際的數字例子，說明它算出來會是多少。

為什麼要加「不要改動邏輯」

不加這一句，AI 常常會順手「幫你優化」。你要的是解釋，不是改寫——改寫之後你讀的就不是原本那份了

為什麼要加「完全不懂程式的人」

不加的話它會用專業術語寫註解（「初始化變數」「解構賦值」），那等於沒解釋

讀完之後要做什麼

挑一行你還是看不懂的，直接追問：「第 N 行那句是什麼意思？舉一個實際的例子」

常見的意外收穫

AI 在寫註解的過程中，有時候會自己發現邏輯有問題並主動說出來

把左邊那段直接複製貼上，把檔名與函式名換成你自己的。這一頁的指令是設計過的——每一句限制都有理由，不要省略。

技巧 ②　要它畫出執行流程

可以直接照抄的指令

① 先貼這一段

請把 `calculate` 函式的執行順序畫成文字流程圖，
包含所有的判斷分支，以及每一種分支會回傳什麼。
用條列的方式，不要用程式碼。

看完它的回答之後，接著問這一段

② 追問

如果使用者「沒有選身分別就直接按計算」，
會走到流程圖的哪一步？那一步會回傳什麼給畫面？

什麼時候用這個

程式碼裡有 if、有迴圈、有多種情況時。單純一路算到底的程式碼不需要

要看什麼

看「有沒有哪一種情況沒有被處理到」。第 2 週的 Bug 3 就是這樣——沒選身分別這條路徑，原本的程式碼根本沒想過

追問句型

「如果使用者做了 X，會走到哪一個分支？」把你想得到的每一種操作都問一次

自己也要畫一次

先自己想一遍「應該有哪幾種情況」，再跟它畫的比對。少掉的那幾種就是漏洞

把左邊那段直接複製貼上，把檔名與函式名換成你自己的。這一頁的指令是設計過的——每一句限制都有理由，不要省略。

技巧 ③　問它邊界問題

可以直接照抄的指令

① 先貼這一段

如果 level 是 1、9、0、-1、或不是數字，calculate 會發生什麼事?
如果 identity 是空字串或不存在的值，又會發生什麼事?
請逐一說明，並指出哪些情況是沒有被妥善處理的。

看完它的回答之後，接著問這一段

② 追問

上面你說「沒有被妥善處理」的那幾種情況，
請各寫一句話說明使用者實際會看到什麼畫面。

為什麼邊界最容易錯

程式邏輯多半是照「正常情況」寫的。最小值、最大值、空值、超出範圍——這四種是錯誤最愛藏的地方

一定要自己再試一次

AI 說「會丟出錯誤」，你要實際去點一次確認。它有時候會答錯，而且答得很有把握

醫療場域的額外提醒

邊界常常剛好是最重症的個案。第 8 級算錯，影響的是需求最高的那群人

這一問的產出

直接就是測試案例。Step 3 會把它們寫成 test.html 裡的邊界測試

把左邊那段直接複製貼上，把檔名與函式名換成你自己的。這一頁的指令是設計過的——每一句限制都有理由，不要省略。

技巧 ④　要它列出所有假設

可以直接照抄的指令

① 先貼這一段

請列出 `calculator.js` 這份程式碼做了哪些假設。
包含對輸入資料的假設、對使用情境的假設、
以及對外部條件（例如法規、額度表）的假設。

看完它的回答之後，接著問這一段

② 追問

上面這些假設裡，哪一個一旦不成立，
會讓試算結果整個錯掉、但畫面上完全看不出來?

為什麼這一問最有價值

假設是「沒有寫出來但必須成立」的東西。它不會出現在程式碼裡，也不會報錯——但假設一旦不成立，整個結果就是錯的

這份程式碼的實際假設

額度表不會改、使用者一定會先選身分別、 level 一定是 2–8 的整數、算的是上限而不是核定金額

哪一個假設最危險

「額度表不會改」。它會隨年度公告調整，而程式碼裡沒有任何機制會提醒你過期了

這一問的產出

直接就是驗證報告第四段與 README 的「已知的限制」。你不必另外想

把左邊那段直接複製貼上，把檔名與函式名換成你自己的。這一頁的指令是設計過的——每一句限制都有理由，不要省略。

審查檢核表：不會寫程式也能逐項檢查

#	檢查什麼	怎麼問 AI ／怎麼自己確認	在這個專案裡
1	數字有沒有寫死	「程式裡有哪些寫死的數字？它們從哪來？」	額度表寫死在 AMOUNTS 。沒問題，但要記錄它會過期
2	空值有沒有處理	「輸入是空的會怎樣？」自己實際留空試一次	身分別留空曾經會直接壞掉（第 2 週的 Bug 3）
3	邊界用 < 還是 <=	「最小值與最大值分別會走到哪一行？」	第 8 級曾經被壓回第 7 級（第 2 週的 Bug 2）
4	錯誤訊息使用者看得懂嗎	故意做錯一次，看畫面上出現什麼	應顯示「請先選擇身分別」，而不是靜默無反應
5	資料會被送到哪裡	「這段程式碼有沒有對外連線？送了什麼出去？」	完全沒有對外連線。所有計算都在瀏覽器內完成
6	有沒有金鑰或密碼	全文搜尋 key 、 token 、 password 、 secret	沒有。本專案不需要任何金鑰
7	算的東西跟需求一致嗎	拿第 1 週寫的規格逐條對照	規格說要顯示四個數字，實際顯示四個。一致
8	有沒有超出應有的範圍	「它有沒有做規格以外的事？」	沒有。但要留意 AI 有時會自作主張加功能

這張表 Step 2 要實際填一遍。填的時候不要只打勾——每一項都要寫下「我是怎麼確認的」。寫不出來，就代表你其實沒檢查，只是覺得應該沒問題。

PART 03

什麼是測試

What a Test Actually Is

你不需要學會寫測試。
但你必須看得懂它在測什麼，以及它「沒有」測到什麼。

一個測試，就是一句「給它這個，應該得到那個」

test.html 裡最簡單的一個測試

```
test('一般戶第 5 級，自負額 3,856', function () {

  expect( calculate(5, 'general').copay ).toBe( 3856 );

});


//  翻成中文就是：
//
//    這個測試叫做「一般戶第 5 級，自負額 3,856」
//    給 calculate 這個函式 5 和 general 這兩個值，
//    它算出來的 copay 應該要等於 3856。
//
//    如果等於 → 顯示綠色的 ○
//    如果不等於 → 顯示紅色的 ×，並寫出實際得到多少
```

1 測試的名字

用中文寫，寫給人看的
名字要寫得讓你三個月後還看得懂它在驗什麼

2 給它什麼 calculate(5, 'general')

輸入：CMS 第 5 級、一般戶
這就是你在畫面上會選的那兩個值

3 應該得到什麼 .toBe(3856)

這個數字從哪來？
24,100 × 16% = 3,856。你自己算的，不是 AI 給的

4 誰決定對錯

是你，不是程式。程式只負責比對
所以測試永遠只能驗到「跟你想的一樣」

這一頁只要記住一件事：測試不是什麼高深的東西，它就是把你原本會手動點一次的動作，寫成一句話讓電腦幫你點。差別在於——電腦可以在三秒內幫你點 29 次，而且永遠不會忘記。

整份 test.html 的結構：三段而已

test.html（節錄）

```
// ① 兩個工具函式（20 行，看過就好，不必懂）
function test(name, fn) { ... }
function expect(actual) { ... }


// ② 你的測試（本週的主體）
test('驗收條件 1｜一般戶第 5 級，自負額 3,856', function () {
  expect(calculate(5, 'general').copay).toBe(3856);
});

test('驗收條件 2｜低收入戶第 5 級，自負額 0', function () {
  expect(calculate(5, 'low').copay).toBe(0);
});

test('邊界｜第 8 級給付上限 36,180', function () {
  expect(calculate(8, 'general').limit).toBe(36180);
});


// ③ 把結果畫到畫面上（20 行，看過就好）
```

1 工具函式

這一段你完全不必懂，也不必改
它只是提供 test() 和 expect() 兩個字讓你用

2 你的測試

本專案 29 項＝驗收條件 4 ＋邊界 4 ＋組合 21
你要看懂、要判斷夠不夠的，只有這一段

3 畫面輸出

把通過與失敗畫成綠色 ○ 與紅色 ×
也不必改

4 為什麼不用 npm

這 20 行就取代了測試框架，代價是功能陽春
好處是雙擊就能跑——這比功能完整重要得多

給 B 軌的補充：正式專案會用 Vitest、Jest 這類框架，語法幾乎一模一樣（也是 test / expect / toBe），差別在於要用 npm 安裝並從終端機執行。你在這裡學到的觀念可以直接搬過去。

為什麼一定要先把計算邏輯拆出來



這是測試最被低估的價值：它除了抓錯之外，還會反過來逼你把程式碼組織好。一份「測不起來」的程式碼，幾乎必然也是一份「改不動」的程式碼——這兩件事是同一個問題的兩面。

先寫測試，還是先寫實作?

比較項目	先寫實作（多數人的做法）	先講清楚怎樣算對
實際順序	① 叫 AI 寫功能 ② 自己點一點 ③ 覺得對了收工	① 先講清楚「怎樣算對」 ② 叫 AI 寫測試 ③ 再叫它寫功能直到測試變綠
誰決定「對」	AI 寫完之後，你才回頭看它對不對——標準被它的產出帶著走	你在它動手之前就先定義好——標準是你的
改東西的時候	改完只能再手動點一次。而且通常只點你剛改的那一塊	改完重跑一次， 29 項全部驗過。三秒
改 A 壞 B	幾乎必然會發生，而且你不會發現	當場變紅，而且明確告訴你是哪一項壞了
要多花的時間	0 分鐘（但之後會用除錯時間還回來）	第一次約 20 分鐘。之後每次重跑是 3 秒
這門課的要求	第 1 、 2 週可以這樣。第 3 週起不行	期末專題「驗證證據」佔 14% ，要的就是這個
實務上的折衷	—	不必每個功能都先寫測試。但「會算錢、會影響決策」的那幾個一定要

對這門課的具體建議：不必變成測試基本教義派。挑出「算錯會有人受影響」的那幾個函式，只為它們先訂驗收條件與測試就夠了。長照試算器裡，那就是 calculate() 這一個。

一條驗收條件，會長出好幾個測試案例

第 1 週寫的驗收條件

「低收入戶不論等級，自負額一律為 0」

一句話，看起來很單純

展開成四種案例

寫進 test.html

① 正常情況 — 最典型的那一種

- 第 5 級 × 低收入戶
- 自負額應為 0

→ test(...) 第 1 項

② 邊界情況 — 最小與最大

- 第 2 級 × 低收入戶 → 0
- 第 8 級 × 低收入戶 → 0

→ test(...) 第 2、3 項

③ 對照組 — 確認別人沒被改壞

- 第 5 級 × 一般戶 → 3,856
- 低收改 0 後一般戶不能跟著變

→ test(...) 第 4 項

④ 全覆蓋 — 所有等級都驗一次

- 7 個等級 × 低收 → 全部 0
- 另外兩種身分別一起驗

→ test(...) 21 項組合

這一頁回答的是「29 項測試哪來的」。你不必自己想 29 項——把 3 條驗收條件各照這四個角度展開，自然就會有二十幾項。而且這四個角度可以直接寫進給 AI 的指令裡（Step 3 會用到）。

PART 04

你以為測完了

You Think You Tested It

「我試過了，沒問題」——通常指的是自己最熟的那一兩種組合。
而錯誤幾乎都藏在沒測的格子裡。

你以為測完了： 21 種組合，你實際測了幾種

身分別 ＼ CMS	2 級	3 級	4 級	5 級	6 級	7 級	8 級
一般戶	○	?	?	○	?	?	?
中低收	?	?	?	?	?	?	?
低收	?	?	?	×	?	?	○

○ 測過，結果正確　× 測過，發現錯誤　? 還沒測過

多數人測了三格就上線

「我試過了，沒問題」——通常指的是自己最熟的那一兩種組合
而錯誤幾乎都藏在沒測的格子裡

這張表點一點只要三分鐘

21 種聽起來很多，實際操作三分鐘跑得完
這三分鐘會抓到大部分的錯

已經抓到的那一格

第 5 級 × 低收：自負額顯示 3,856
低收戶的自負額比率是 0%，應該顯示 0
這種錯不會報錯，程式跑得好好的

哪幾格最該優先測

四個角落（最低級／最高級 × 最寬鬆／最嚴格身分別）
邊界值最容易錯——查表用 < 還是 <= 差一格

這一頁要建立的不是「要很認真測」的態度，而是一個具體動作：把所有輸入組合列成一張表，一格一格點過去，並把結果標上去。這張表本身就是你交出去的驗證證據——它比一句「我測過了」有說服力得多。

五個一定要自己試的情境（不能只靠測試）

#	情境	在這個專案裡怎麼試	為什麼測試蓋不到
1	邊界值	第 2 級（最低）與第 8 級（最高）各試一次	測試蓋得到，但你要先想得到「有邊界」這件事才寫得出來
2	每一種組合	7 × 3 ＝ 21 種，照網格一格一格點	測試蓋得到。但用手點過一次，你才會注意到畫面上的問題
3	空輸入 / 不選	身分別留在「請選擇」直接按計算	測試只驗「有沒有丟出錯誤」，驗不到「使用者看到的訊息夠不夠清楚」
4	重複操作	連按十次計算、來回切換等級、算完再改回不選	測試通常只跑一次。第 2 週那個「維持舊數字」的 bug 就只有這樣才試得出來
5	換一個裝置	用手機開一次；用別人的電腦開一次	測試完全蓋不到版面。本機好好的、手機破版是常態

一個常見的誤解：「有測試了就不用自己點」。剛好相反——測試負責「重複驗證數字」，人負責「發現數字以外的問題」。第 3、4、5 項是測試永遠取代不了的，而它們恰好是使用者最先感受到的。

時間不夠的話，先測這四個角落

身分別 ＼ CMS	2 級	3 級	4 級	5 級	6 級	7 級	8 級
一般戶	○	?	?	?	?	?	○
中低收	?	?	?	?	?	?	?
低收	○	?	?	?	?	?	○

○ 測過，結果正確　　× 測過，發現錯誤　　? 還沒測過

為什麼是四個角落

最低與最高等級　×　最寬鬆與最嚴格身分別
四個角落＝ 2 級／ 8 級　×　一般戶／低收

錯誤最愛藏在這裡

查表用 < 還是 <= ，差的就是最高那一格
比率是 0 的那一種，最容易被寫成別的數字
第 2 週的兩個 bug 都落在角落

中間那些格子呢

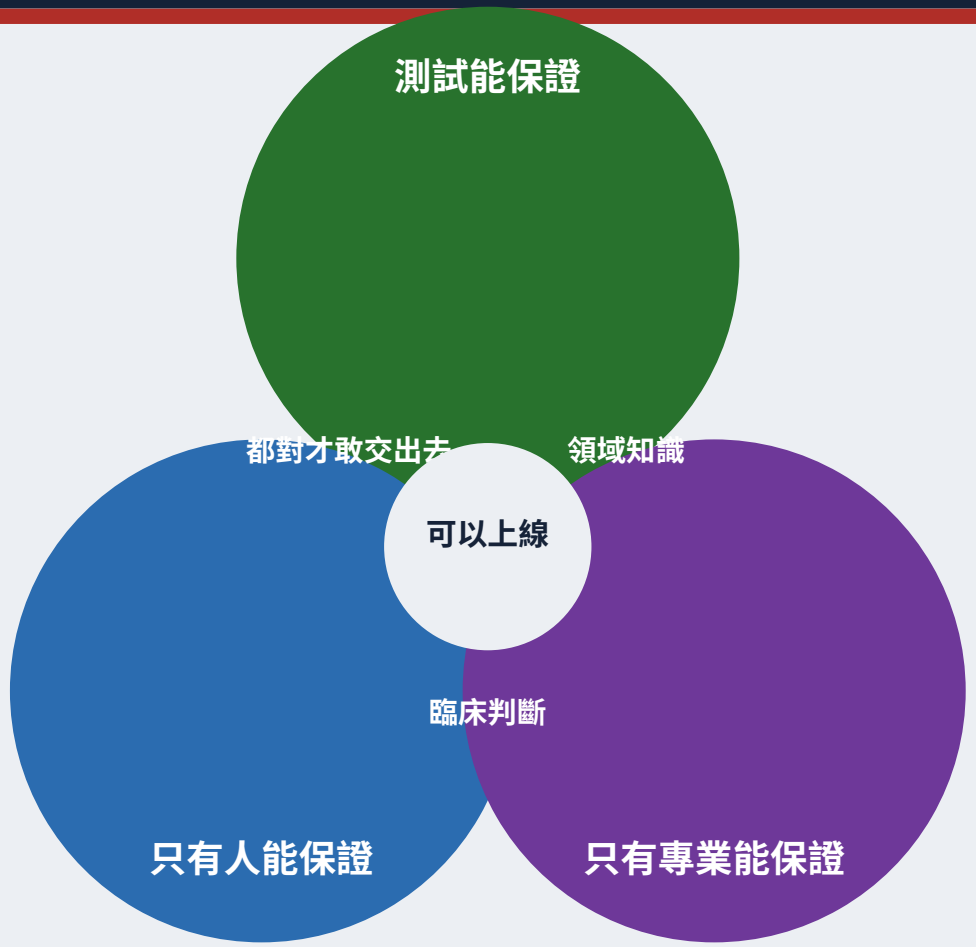
中間的格子多半是「同一套邏輯重複跑」
角落對了，中間通常也會對
但這只是省時間的權宜，不是可以不測

醫療場域的額外理由

最高等級＝失能最嚴重的個案
那一格算錯，影響的是需求最高、最無法自行察覺的人

這一頁是給時間真的不夠的時候用的權宜做法，不是標準做法。本週的作業仍然要求 21 格全填——因為你只有 21 格，三分鐘就點完了。等你的專案有 200 種組合時，才需要用這個策略。

測試能保證的，與只有人能保證的



測試能保證

- 算出來的數字
- 跟你寫的期望一致

- 而且改完之後
- 還是一致

只有人能保證

- 期望本身是對的
- （24,100 × 16% ＝ 3,856
  這件事是你查出來的）

- 使用情境合不合理

只有專業能保證

- 這個數字給家屬看
- 會不會被誤解

- 什麼情況下
- 根本不該用這個工具

這一頁是本週最重要的界線：測試綠了，只代表「程式照你想的做」。至於「你想的對不對」「這樣做合不合適」，測試完全無能為力。涉及給付金額、用藥、臨床決策的部分，一定要有懂的人看過一次——這一條在期末專題的評分規準裡是硬性要求。

PART 05

動手做： Step 1–7

Hands-on

從這裡開始請打開你自己的專案，跟著做。

每一步都有「你應該看到什麼」——沒看到就先停下來，不要往下走。

Step 1　要 AI 把核心計算逐行加上中文註解

你現在要做什麼　▶　打開你的 calculator.js，把下面這段貼進 AI 的對話框

貼進 AI 的對話框

請把 calculator.js 的 calculate 函式逐行加上中文註解，
用完全不懂程式的人也看得懂的話解釋每一行在做什麼。
不要改動任何邏輯，只加註解。

另外請在最後補一段：這個函式做了哪些「沒有寫出來
但必須成立」的假設？

你應該看到什麼

AI 回給你一份跟原本一模一樣的程式碼，
但每一行上面多了一句中文說明。
行數應該跟原本差不多（只多了註解），
如果它回給你一份看起來很不一樣的程式碼，
代表它順手改寫了——重新要一次。
最後會多出一段「假設」清單，
通常有 3–5 條。

沒看到的話，先檢查這幾件事

① 它把邏輯也改掉了
→ 重貼，加「一個字都不要改」
② 註解出現「初始化」「解構」等術語
→ 追問「請用日常的話重寫」
③ 它問你要哪一個函式
→ 整份 calculator.js 貼給它

這一步不要急。你接下來所有的判斷，都建立在「你真的知道它在算什麼」之上。讀不懂就一直追問到懂為止——這不是浪費時間，這就是本週的主體。

帶讀 ①　額度表那一段在做什麼

calculator.js （節錄）

```
const AMOUNTS = [
  0,        // 0（不使用）
  0,        // 1 級（不核發照顧及專業服務給付）
  10020,    // 2 級
  15460,    // 3 級
  18580,    // 4 級
  24100,    // 5 級
  28070,    // 6 級
  32090,    // 7 級
  36180,    // 8 級
];
```

1　這是一張查詢表

把 CMS 等級對應到給付上限，就這樣而已
跟 Excel 裡的一欄對照表是同一件事

2　為什麼前兩格是 0

這是用「位置」當等級：第 2 格放 2 級的金額
第 0、1 格用不到，但必須佔位，否則位置就對不上了

3　你要檢查什麼

把這七個數字跟長照給付基準逐一核對，並記下出處與查證日期
這是你的專業，不是 AI 的——它沒查過

4　這裡有一個假設

假設「額度表不會改」——但它會隨年度調整，而程式不會提醒你
→ 寫進驗證報告第四段

讀程式碼的第一個心得通常是：「原來就這樣？」——對，就這樣。多數業務邏輯翻成人話之後都非常單純，複雜的是語法，不是邏輯。而語法你可以不必懂。

帶讀 ②　查表那一段在做什麼

calculator.js （節錄）

```
const MIN_LEVEL = 2;
const MAX_LEVEL = 8;

function lookupPayment(level) {

  if (level < MIN_LEVEL || level > MAX_LEVEL) {
    throw new Error('CMS 等級必須介於 2 到 8 之間… ');
  }

  return AMOUNTS[level];
}
```

1　先擋掉不合理的輸入

低於 2 或高於 8 就直接丟出錯誤，不往下算
「丟出錯誤」＝停下來並說明原因

2　邊界用的是哪一種

`< 2` 和 `> 8`—— 所以 2 和 8 本身是「合法」的
這一格就是第 2 週 Bug 2 的位置。當初寫成限制在 7

3　再去查表

level 是 5 就回傳 AMOUNTS 的第 5 格 ＝ 24100
一行而已

4　自己驗一次

實際去畫面上選第 2 級與第 8 級各一次
看到 10,020 與 36,180 才算過

注意第 2 點：用 MIN_LEVEL ／ MAX_LEVEL 這種有名字的常數，比直接寫 2 和 8 好——因為改的時候只要改一個地方，而且看得出它的意思。
這是你可以向 AI 提出的要求。

帶讀 ③　主要計算那一段在做什麼

calculator.js （節錄）

```
function calculate(level, identity) {

  const info = IDENTITY_RATES[identity];
  if (!info) {
    throw new Error('請先選擇身分別');
  }

  const limit = lookupPayment(level);
  const copay = Math.round(limit * info.rate);
  const subsidy = limit - copay;

  return { level, identityName: info.name,
           limit, copayRate: info.rate,
           copay, subsidy };
}
```

1　先查身分別

查不到就停下來說「請先選擇身分別」
這是第 2 週 Bug 3 修好之後的樣子

2　算錢就這三行

查上限 → 上限 × 比率 = 自負額 → 上限 − 自負額 = 補助
整個試算器的核心，就是這三行

3　Math.round 是四捨五入

24100 × 0.16 = 3856 剛好整數，但別的組合有小數
四捨五入對不對？實際給付怎麼進位要查規定

4　回傳一包數字

把六個值打包給畫面用
這裡沒有計算，只是整理

第 3 點是一個很好的例子：程式碼寫得完全正確，但「四捨五入」這個決定對不對，只有懂長照給付規則的人能判斷。這就是你在這件事上比工程師強的地方——請把它寫進驗證報告。

Step 1-2　追問，直到每一行你都能用自己的話說出來

你現在要做什麼　▶　挑出你還是看不懂的那一行，用下面的句型追問

追問句型（三選一）

① 第 N 行「〔貼上那一行〕」是什麼意思?
請舉一個實際的數字例子說明。

② 如果 identity 是空字串，程式會走到哪一行?
使用者會看到什麼?

③ 這一段有沒有哪一種輸入是沒有被處理到的?

你應該看到什麼

它會用更白話的方式重講一次，
而且通常會舉一個帶數字的例子。
判斷你懂了沒有的標準：
你能不能不看程式碼，
用自己的話跟同學說出
「這個試算器是怎麼算的」。
說得出來， Step 1 就完成了。

沒看到的話，先檢查這幾件事

① 又是一堆術語
→ 加「當作我完全沒學過程式」
② 它前後說法不一致
→ 多半在猜。重貼整份再問
③ 它講的跟畫面行為不一樣
→ 相信畫面，實際點一次

一個很好用的自我檢查：把 calculator.js 關掉，然後試著跟別人解釋「第 5 級低收入戶為什麼自負額是 0」。講得出來就是真的懂了；講不出來，回去再問幾次。

Step 2　用審查檢核表逐項檢查

你現在要做什麼　▶　打開第 15 頁的檢核表，八項逐一檢查並寫下你怎麼確認的

八項檢查（第 15 頁）

1　數字有沒有寫死
2　空值有沒有處理
3　邊界用 <　還是 <=
4　錯誤訊息使用者看得懂嗎
5　資料會被送到哪裡
6　有沒有金鑰或密碼
7　算的東西跟需求一致嗎
8　有沒有超出應有的範圍

每一項都要寫「怎麼確認的」

不是打勾就好。例：
第 3 項　○　實際選第 2 級與第 8 級各試一次，
顯示 10,020 與 36,180，正確

你應該看到什麼

八項都有一句「我怎麼確認的」。
而且至少有一項被你標成「有問題」。
如果八項全部沒問題，
高機率是你檢查得不夠仔細——
回去看第 1 項（數字寫死）與
第 7 項（跟需求一致）再想一次。

沒看到的話，先檢查這幾件事

① 不知道怎麼確認某一項
→ 把該項的問法貼給 AI 問
② 找不到任何問題
→ 做第 35 頁的三個追問
③ 覺得很花時間
→ 約 20 分鐘，是本週最有價值的部分

為什麼「至少要找到一個問題」是硬性要求：任何一份程式碼都有可以改進的地方。找不到，代表你還沒學會怎麼看——這一項在期末評分規準裡也是同樣的標準。

審查檢核表：一份填好的完整範例

#	檢查什麼	判斷	我是怎麼確認的
1	數字有沒有寫死	○ 有，但可接受	七個數字寫死。已對照衛福部給付基準核對正確；但會隨年度調整且無過期提醒 → 列入不確定
2	空值有沒有處理	○ 有	身分別留「請選擇」按計算，畫面出現「請先選擇身分別」，不會靜默無反應
3	邊界用 < 還是 <=	○ 正確	實際選第 2 級與第 8 級各一次，顯示 10,020 與 36,180，與基準相符
4	錯誤訊息看得懂嗎	△ 勉強	訊息看得懂，但沒把該選的欄位標出來，高齡使用者不知道往哪看 → 可改進
5	資料會送到哪裡	○ 沒有外送	全文搜尋 fetch、XMLHttpRequest、axios 都沒有。所有計算在瀏覽器內完成
6	有沒有金鑰密碼	○ 沒有	全文搜尋 key、token、password、secret 都沒有。本專案不需要金鑰
7	跟需求一致嗎	○ 一致	對照第 1 週規格：輸入兩項、輸出四項，實際完全相符
8	有沒有超出範圍	× 有問題	AI 自己加了「僅供參考」說明，規格沒要求。內容是好的，但代表它會自作主張

注意第 4 與第 8 項：一份誠實的檢核表一定會有「△」與「×」。全部都是「○」的檢核表，讀的人第一個反應是「他大概沒認真檢查」——這在期末評分時也是一樣的。

Step 2-2　找不到問題的話，試這三個追問

你現在要做什麼　▶　把下面三段依序貼給 AI，通常會逼出你自己沒想到的東西

① 逼出假設

請列出這份程式碼做了哪些「沒有寫出來但必須成立」的假設。
每一條請說明：如果它不成立，使用者會看到什麼？

② 逼出遺漏

如果我是一個完全不懂長照的家屬，第一次打開這個頁面，
我最可能誤解什麼？

③ 逼出風險

如果這個工具明天要給 500 位照管專員實際使用，
你最擔心哪三件事？請具體說明。

你應該看到什麼

每一問都會生出 3–5 條。
你要做的不是全盤接受，
而是逐條判斷：
「這一條在我的情境下成不成立？」
成立的挑出來，寫進檢核表
與驗證報告第四段。

沒看到的話，先檢查這幾件事

① 太籠統（「要注意資安」）
→ 追問「具體是哪一行、會發生什麼」
② 列了一堆看不懂的技術風險
→ 多半不適用於小工具。問
「這在單頁靜態網頁會發生嗎」

第 ② 問特別值得做。它把視角從「程式對不對」換成「人會不會誤解」——而在醫療場域，後者造成的傷害往往更大。第 2 週那個「維持舊數字」的行為，就是典型的誤解來源。

Step 3　把驗收條件請 AI 寫成測試

你現在要做什麼　▶　把第 1 週的驗收條件，連同下面的指令一起貼給 AI

貼進 AI 的對話框（把〔〕換成你自己的）

請幫我寫一份 test.html，測試 calculator.js 的 calculate 函式。

要求：
1. 不要用 npm 或任何框架。用瀏覽器打開就能跑。
2. 自己寫一個最簡單的 test() 與 expect().toBe()。
3. 通過顯示綠色，失敗顯示紅色並寫出預期值與實際值。
4. 要測的項目：
〔貼上你第 1 週寫的 3 條驗收條件〕
另外加上邊界（最低 2 級、最高 8 級、超出範圍）
以及全部 21 種組合（7 個等級 × 3 種身分別）。
5. 每個測試的名字用中文寫，寫清楚在驗什麼。

你應該看到什麼

它會給你一份完整的 test.html。
存到跟 calculator.js 同一個資料夾，
檔名就叫 test.html。
打開來大致掃一遍，
確認裡面有 <script src="calculator.js">
——沒有的話它會找不到你的函式。

沒看到的話，先檢查這幾件事

① 給的是需要 npm 的版本
→ 強調「要能雙擊 html 直接跑」
② 把 calculator.js 內容複製進去了
→ 不行，要用 <script src> 引用，
否則改了程式碼測試不會跟著變
③ 測試名字是英文 → 要求改中文

第 ② 個陷阱很重要也很常見：如果測試裡放的是「複製過去的一份程式碼」，那你測的就不是你實際在用的那一份。改完程式碼測試照樣全綠，完全失去意義。

Step 3-2　跑測試：雙擊 test.html

你現在要做什麼　▶　用瀏覽器打開 test.html （直接雙擊檔案就可以）

A 軌（瀏覽器工具）

在工具的檔案清單裡找到 `test.html`，
點「預覽 / Preview」或「開啟」。

B 軌（本機）

在檔案總管裡找到 `test.html`，直接雙擊。
或用瀏覽器 → 檔案 → 開啟舊檔。

網址列會顯示 `file:///...` 開頭，這是正常的。

你應該看到什麼

最上面一條綠色的橫幅：
　　◯ 全部通過： 29 / 29
下面 29 項每一項前面都是綠色的 ◯。

這就是本週第二個交付物——
截圖存下來，驗證報告要用。

沒看到的話，先檢查這幾件事

① 整頁空白
　→ 兩個檔要在同一個資料夾
② 顯示「calculate is not defined」
　→ <script src="calculator.js"> 漏了
　　或檔名拼錯
③ 有紅色的 × → 很好，看下一頁

第一次跑就全綠不一定是好事——有可能是測試寫得太寬鬆。建議故意把 calculator.js 裡低收的 rate 從 0 改成 0.05，重新整理，確認它真的會變紅。會變紅，才代表測試真的在測東西。

測試變紅的時候，怎麼讀那一行字

紅色那一行

畫面上顯示的
× 驗收條件 2 ｜低收 5 級
預期 0，實際得到 1205
（紅色底，排在清單裡）

四個步驟

怎麼拆解
① 這一項在驗什麼 → 看名字
② 我以為會是多少 → 預期 0
③ 實際跑出來多少 → 1205
④ 誰錯了？先判斷這一件事

先問「誰錯了」，不要急著改程式

有兩種可能：程式算錯了，或是測試的預期值寫錯了。先自己手算一次（24,100 × 0% ＝ 0），確認誰對

這個例子裡是程式錯

低收入戶部分負擔為 0%，所以預期 0 是對的， 1205 是程式算錯 → 改 calculator.js

千萬不要為了變綠去改測試

把預期值從 0 改成 1205，畫面確實會變綠——但你只是把錯誤合法化了。這是驗證裡最嚴重的自我欺騙

改完之後

重新整理 test.html，確認變綠，而且其他 28 項沒有跟著變紅

紅色不是壞消息，是好消息——它代表測試正在做它該做的事。真正的壞消息是「改壞了卻還是全綠」。

Step 3-3　故意改壞，確認測試真的在把關

你現在要做什麼　▶　把 calculator.js 裡低收入戶的 rate 從 0 改回 0.05，存檔，重新整理 test.html

改這一行

```
const IDENTITY_RATES = {
  general: { name: '一般戶', rate: 0.16 },
  midlow:  { name: '中低收入戶', rate: 0.05 },
  low:     { name: '低收入戶', rate: 0.05 },   // ← 改成 0.05
};
```

確認完之後記得改回來

```
  low:     { name: '低收入戶', rate: 0 },
```

你應該看到什麼

紅色的橫幅：
× 有 9 項沒過（20 / 29 通過）
每一項紅色下面都寫著
「預期 0，實際得到 1205」。
你親眼看到「測試會擋住改壞的東西」——
沒做過這一步的人不會相信它有用。

沒看到的話，先檢查這幾件事

① 改了卻還是全綠
→ 嚴重問題。代表 test.html 裡放的是複製過去的程式碼，不是引用的。回到 Step 3 重做

② 忘記改回來 → 交作業前一定要確認是全綠的狀態

這一步是本週最容易被跳過、卻最重要的一步。它回答的是：「我怎麼知道我的測試不是擺著好看的？」——答案是故意弄壞一次。這個習慣在往後的每一個專案都用得上。

Step 4　填 21 格覆蓋網格

你現在要做什麼　▶　打開你的試算器畫面，7 個等級 × 3 種身分別，一格一格點過去

怎麼填

○　點過了，數字跟你算的一樣
×　點過了，數字不對（記下實際看到什麼）
?　還沒點

自己算一次：上限 × 比率 = 自負額
　一般戶 16%、中低收 5%、低收 0%

可以直接抄的空白表

	2 級	3 級	4 級	5 級	6 級	7 級	8 級
一般戶	?	?	?	?	?	?	?
中低收	?	?	?	?	?	?	?
低收	?	?	?	?	?	?	?

你應該看到什麼

21 格全部變成 ○ 或 ×。
一格都不能留？。
實際操作大約三分鐘。
這張填好的表就是交付物之一，
拍照或截圖存下來。

沒看到的話，先檢查這幾件事

① 「測試測過了，還要點嗎」
　→ 要。對齊跑掉、金額變 NaN、
　　標點怪異——測試都看不到
② 有一格對不上
　→ 標 ×，記下實際看到的數字，
　　這就是驗證報告的素材

為什麼手點過一次不能省：測試只檢查 calculate() 回傳的數字。從「數字」到「使用者眼睛看到的畫面」中間還有一整層——格式化、對齊、單位、標點。那一層只有人看得到。

填完之後，這樣讀你的網格

情況	代表什麼	接下來要做什麼
全部 ○	恭喜，但先別高興太早	回頭確認：你算的「應該是多少」是對的嗎？ 去查一次給付基準，不要用 AI 給的數字當標準答案
有 1–2 個 ×， 分散在不同地方	多半是個別的計算或顯示問題	一個一個修，每修完重跑一次測試 + 重點那一格
整列或整欄都是 ×	某一種身分別或某一段等級的邏輯整個錯了	不要一格一格修。回到 calculator.js 看那一段， 通常是一個地方錯導致整排錯
角落是 ×， 中間都對	典型的邊界問題（< 與 <= 之類）	第 2 週 Bug 2 就是這樣。檢查 lookupPayment 的範圍判斷
點的時候發現 畫面怪怪的	版面、格式、單位的問題	記下來。測試抓不到這些，但使用者第一眼就會看到

> 第一列值得多說一句：「全部 ○」最大的風險是「你的標準本身就是錯的」。所以驗證報告要寫「我的預期值從哪裡來」——寫「我查了衛福部的給付基準」跟寫「AI 說的」，可信度完全不同。

Step 5　寫驗證報告：四段

你現在要做什麼　▶　照下面的四段各寫 3-5 句，總共約一頁

四段各寫什麼

① 測了什麼
驗收條件幾條、測試幾項、網格幾格、自己點了幾種情境

② 怎麼測的
預期值從哪來？（這一句最重要）

③ 抓到什麼
發現了哪些問題、怎麼修的、修完怎麼確認

④ 還有哪些不確定
我沒驗到的、我不確定的、什麼情況不該用它

你應該看到什麼

一份約一頁的文件，四段齊全。
第四段不能留白，
而且不能只寫「沒有」。
留白視為未通過——
說得出自己不知道什麼，
是本週的核心能力。

沒看到的話，先檢查這幾件事

① 第四段真的想不出來
→ 回去看 Step 1 最後 AI 列的「假設」清單，
以及 Step 2-2 的三個追問

② 不知道要寫多細
→ 標準是「另一個人讀完，
知道能不能相信這個工具」

這份報告是期末專題五項交付物之一，也是評分佔比最高的一項（14%）。現在寫的這一份，格式與深度就是期末的標準——留著，期末直接改寫。

驗證報告範例（上）：測了什麼、怎麼測的

段	內容
① **測了什麼**	依第 1 週規格的 3 條驗收條件，寫成 test.html 共 29 項自動測試 （驗收條件 4 項、邊界值 4 項、全部 21 種組合）。 另以人工實際操作 21 種組合各一次，並在手機尺寸（390px）下檢視版面。
② **怎麼測的**	預期值來源：衛生福利部「長期照顧服務申請及給付辦法」照顧及專業服務給付額度 （CMS 2–8 級 10,020 ～ 36,180 元），部分負擔比率一般戶 16%、中低收 5%、低收 0%。 自負額由我自行以「上限 × 比率」計算後才寫進測試——不是採用 AI 給的數字。 並曾故意將低收比率改為 0.05，確認測試由 29/29 變為 20/29，證明測試確實有把關。

第 ② 段是整份報告最關鍵的一段。「預期值從哪來」決定了這份驗證有沒有意義——寫「我查了衛福部的給付基準」跟寫「AI 算給我的」，可信度天差地遠。

驗證報告範例（下）：抓到什麼、還有哪些不確定

段	內容
③ 抓到什麼	① 第 5 級 × 低收入戶自負額顯示 1,205（應為 0）——比率誤植，已修正。 ② 第 8 級給付上限顯示 32,090（應為 36,180）——範圍判斷 off-by-one，已修正。 ③ 未選身分別時畫面靜默無反應——已改為顯示「請先選擇身分別」。 三項修正後 29 項測試全數通過，並重新人工複驗 21 格。
④ 還有哪些不確定	① 給付額度寫死在程式碼中，年度公告調整時不會自動更新，沒有任何過期提醒機制。 ② 只計算「照顧及專業服務」，未含交通接送、輔具、喘息——使用者可能誤以為是全部。 ③ 顯示的是額度上限而非核定金額，家屬可能誤解為「我可以領到這麼多」。 ④ 四捨五入是否符合給付規定、無障礙（鍵盤與色彩對比）皆未查證。 ⑤ 未經專業單位審核，介面上的「僅供參考」標示不得移除。

第 ④ 段列了五條，這是合理的數量。少於三條通常代表想得不夠——任何一個工具都有三件以上不確定的事。這一段寫得越誠實，讀的人越能判斷要不要相信它。

Step 6　寫完之後，先回答這四個問題再交

Q 1

你的預期值是從哪裡來的?

如果答案是「AI 算給我的」——那你驗的只是它跟自己一致。回去查一次官方的給付基準

Q 2

如果現在把 calculator.js 隨便改壞一行，測試會變紅嗎?

Step 3-3 做過的話你已經知道答案。沒做過的，現在補做一次

Q 3

21 格裡，你「實際用眼睛看過」的有幾格?

測試跑過不等於你看過。版面、格式、單位的問題只有眼睛抓得到

Q 4

驗證報告第四段，你寫了幾條?

少於三條通常代表想得不夠。任何一個工具都有三件以上不確定的事

這四題不是形式。它們分別對應本週的四個容易造假的地方：標準來源、測試有效性、實際檢視、誠實程度。四題都答得出來，這一週才算真的做完。

Step 7　同儕互審（自學者可自審或請人幫忙）

你現在要做什麼　▶　兩人交換網址與程式碼，用第 15 頁的檢核表審對方的

要交出的三點回饋

① 一個做得好的　（具體，不要只說「很好」）
② 一個我看不懂的　（哪一頁、哪個數字、哪個字）
③ 一個我會擔心的　（如果真的拿去用，我擔心什麼）

自學者的替代做法

找一位家人或同事，請他：
① 不要問你任何問題，自己打開來用一次
② 說出他覺得這個數字是什麼意思

他的理解跟你的設計不一樣的地方，就是問題所在

你應該看到什麼

三點回饋，每一點都具體到
「哪一頁、哪一個數字、哪一句話」。
收到回饋之後不必馬上改，
但要判斷：
「這是他不熟悉，
　還是我真的沒做好？」
判斷結果寫進驗證報告第四段。

沒看到的話，先檢查這幾件事

① 對方只說「做得不錯」
　→ 追問「哪裡你看了要想一下？」
② 你覺得對方的意見不對
　→ 先別急著解釋，
　　「他會誤解」本身就是一個發現
③ 找不到人 → 隔一天自己再看一次

互審最大的價值不是找出 bug，而是發現「原來別人是這樣理解我做的東西」。你太熟悉自己的專案了，熟到看不見它哪裡令人困惑——而使用者第一次打開時，正是那種狀態。

卡住了？照症狀查（一）測試與程式碼

症狀	可能原因	怎麼確認	怎麼修
打開 test.html 整頁空白	找不到 calculator.js	按 F12 看 Console 有沒有紅字	確認兩個檔在同一個資料夾，且檔名大小寫完全一致
顯示 calculate is not defined	沒有引用 calculator.js，或引用在測試之後	看 test.html 裡有沒有 <script src="calculator.js">	這一行要放在測試那一段的前面
改壞程式碼，測試還是全綠	test.html 裡放的是複製過去的程式碼	在 test.html 裡搜尋「function calculate」	搜得到就是複製的。刪掉，改用 <script src> 引用
某一項紅色，但你覺得程式是對的	測試的預期值寫錯了	自己手算一次（上限 × 比率）	先確認誰對。是測試錯就改測試，不要為了變綠去改程式
整排都是紅的	某一段邏輯整個錯，或檔案沒存檔	先存檔再重新整理一次	還是紅的話回到 calculator.js 看那一段
畫面正常，但測試跑不動	瀏覽器擋住了本機檔案互相載入	看 Console 有沒有 CORS 或 file:// 字樣	換 Chrome 或 Edge 試；或把專案先部署上去再跑

使用方式：不必從頭讀。卡住的時候先看「症狀」那一欄找到你的情況，再照「怎麼確認」實際做一次——確認清楚是哪一種原因，再動手修。直接跳到「怎麼修」常常會修錯地方。

卡住了？照症狀查（二） AI 與流程

症狀	可能原因	怎麼修
AI 加註解時把程式也改了	沒有明確禁止它改	重貼一次，指令最後加「只加註解，一個字都不要改」。 並用原本那份檔案，不要用它改過的
AI 的註解還是看不懂	它預設你懂術語	追問：「請當作我完全沒學過程式，用日常的話重寫」。 追問三次還是不懂，就換個問法問「舉一個實際數字的例子」
AI 前後說法不一致	對話太長，它已經 記不住前面的內容	開一個新對話，把完整的 calculator.js 重貼一次再問。 這是 M7 講的上下文長度限制
AI 說「看起來沒問題」	問法太籠統	換成具體的問法：「如果 identity 是空字串會怎樣？」 「請列出這段做了哪些假設？」——具體問題才有具體答案
審查檢核表八項都沒問題	檢查得不夠仔細	做第 35 頁的三個追問。特別是第 ② 問 （家屬第一次打開會誤解什麼）
驗證報告第四段寫不出來	把「不確定」當成缺點	觀念要改：說得出不確定是能力，不是弱點。 素材在 Step 1 的假設清單與 Step 2-2 的追問裡

最後一列值得強調：很多人不寫第四段，是因為覺得「承認有不確定的地方會被扣分」。這門課剛好相反——第四段留白直接視為未通過，因為那代表你不知道自己不知道什麼。

常見的四個誤解（做完之後回頭看一次）

誤解	為什麼是錯的	正確的做法
① 測試全綠 就是對的	測試只證明「程式照你想的做」， 不證明「你想的是對的」	如果預期值是 AI 給的，整套測試只是在驗它跟自己一致。 → 驗證報告第二段一定要寫「預期值從哪來」
② 有測試 就不用自己點	測試驗數字，眼睛驗畫面。 對齊、單位、標點測試全看不到	21 格手點一次只要三分鐘，不要省。 → 使用者第一眼看到的是畫面，不是數字
③ 找不到問題 代表做得很好	任何程式碼都有可改進處。 找不到多半是不知道要找什麼	檢核表八項全過，就去做那三個追問。 → 期末評分同樣要求至少標出一項「有問題」
④ 寫出不確定 會被扣分	完全相反。第四段留白 直接視為未通過	「我不知道 X」是可以處理的資訊，「應該沒問題」不是。 → 使用者需要知道什麼時候不該相信這個工具

這四個誤解有一個共同點：它們都讓人「感覺已經驗證過了」，但實際上沒有。驗證最危險的失敗樣態不是漏掉某一項，而是產生了不該有的安心感。

本週自我檢核（答不出來請回去看標註的頁數）

Q1 **不看程式碼，你能用自己的話說出「第 5 級低收入戶為什麼自負額是 0」嗎？**

答不出來 → 回到第 28–32 頁的 Step 1 與三張帶讀

Q2 **一個測試由哪三個部分組成？「應該得到什麼」那個數字是誰決定的？**

答不出來 → 回到第 17 頁

Q3 **你怎麼知道你的測試不是擺著好看的？**

答案是「故意改壞一次看它會不會變紅」→ 第 39 頁的 Step 3-3

Q4 **測試能保證什麼？不能保證什麼？驗證報告第四段為什麼不能留白？**

答不出來 → 回到文氏圖那一頁與「常見誤解」那一頁

建議做法：不要用看的，實際講出來或寫下來。腦中覺得「大概知道」跟真的說得出口，中間的落差比你想的大——這也是為什麼本週安排了同儕互審。

本週交付與驗收條件

交付物	驗收條件（全部達成才算通過）	怎麼查
① **審查檢核表**	八項都有一句「我怎麼確認的」， 且至少一項標為「有問題」並寫出怎麼處理	文件檢查
② **test.html**	可執行，涵蓋第 1 週訂的 3 條驗收條件、 邊界值、以及 21 種組合	當場跑一次； 自學者對照 範例 \v0.3
③ **覆蓋網格**	21 格全部填完，一格都不留？	看填好的表
④ **驗證報告**	四段齊全。「還有哪些不確定」不得留白， 且不得只寫「沒有」	留白視為未通過
⑤ **同儕回饋**	三點式回饋一份（做得好的／看不懂的／會擔心的）	交換紀錄

這五項裡，① 與 ④ 是期末專題會再用一次的。現在的格式與深度就是期末的標準，做完請留著——期末直接在這個基礎上改寫，不必重來。

第 3 週重點回顧

1. 驗證有三層：讀懂 → 測試 → 說出不確定。缺一層都不叫驗證過
2. 抓錯不需要看得懂程式碼，需要的是「知道正確答案應該是什麼」
3. 一個測試就是一句「給它這個，應該得到那個」。誰決定「應該」？是你
4. 故意改壞一次，才知道測試不是擺著好看的
5. 測試驗數字，眼睛驗畫面。21 格手點一次只要三分鐘
6. 說得出自己不知道什麼，才叫真的驗證過

下一步　下一週：版本控制與上線——AI 改壞了怎麼回去，以及怎麼把東西部署成別人打得開的網址。